University of Bristol

Coursework Report

HeroBot and the MazeDungeon

By

ILIAS ARGYRAKIS

School of Engineering Mathematics and Technology
UNIVERSITY OF BRISTOL

Student:	Ilias Argyrakis
Unit Code:	SEMTM0016
Unit Title:	Artificial Intelligence for Robotics 1
Date:	April 29, 2026

BRISTOL, UNITED KINGDOM, APRIL 2026

Author's Note

The work presented in this report was completed by **Ilias Argyrakis**, a student of the MSc in Aerial Robotics at the University of Bristol. Before arriving at Bristol, I completed a five-year Integrated Master's degree in Computer Engineering and Informatics at the University of Patras (2020–2025), where the final two years were dedicated to a specialisation in computer vision and artificial intelligence. My undergraduate thesis, supervised by Prof. Emmanouil Psarakis, developed a deep learning pipeline for wildfire risk assessment from UAV imagery: a U-Net semantic segmentation model trained on a self-created and hand-annotated dataset that transforms raw aerial images into spatially resolved risk maps. That work required building every stage of the pipeline from scratch; data collection, annotation, model design, training and evaluation. That is the foundation from which I approach machine learning problems today.

Beyond research, I have a genuine interest in teaching. In the final year of my undergraduate degree, I served as a DSP Lab Tutor at the University of Patras, delivering lab lectures on Digital Signal Processing, presenting exercises, guiding students through implementations and fielding their questions in real time. This was not a requirement of my degree; I sought it out because I believe that the ability to explain a concept clearly is the deepest test of whether you have actually understood it. That belief has shaped how I approach technical work ever since: if I cannot teach it, I do not fully know it.

It is with this background in mind that I approach the present assignment. The core requirements; two algorithms per paradigm across three tasks, fall within territory I have covered extensively, both academically and in applied research. Rather than treat this as a constraint, I chose to use it as an opportunity. Alongside the required deliverables, I designed and implemented an enhanced, interactive version of the MazeDungeon environment: a tool with a richer interface, real-time parameter controls, step-by-step decision logs, a visual perception panel and both manual and replay modes. The motivation is straightforward. During my own introduction to neural networks, I found interactive visualisation tools suggested by the lecturers for Neural Networks, those that let you adjust weights, watch information flow and see the effect of your choices immediately, to be far more effective than any lecture or textbook. I wanted to build something equivalent for reinforcement learning and perception: a platform that a lecturer could open in front of a class, hand to a student and use to make abstract concepts tangible. This is not a polished product; it is a proof of concept built in parallel with the research itself. But the intent is sincere and the implementation is functional.

This note is included not to preface the work with excuses, but to provide honest context. The decisions made throughout, which algorithms were chosen, how experiments were designed, where the scope was deliberately extended, reflect a level of prior exposure that goes beyond what this unit assumes. Where that prior exposure informed a choice, it is acknowledged. Where the work goes beyond the brief, it is clearly marked as such. The report that follows does follows the core instructions for the presentation of the assignment for this module, but enhances on certain sections for justified reasons. Integrating code in the report, according to the assignment instructions, reduced the quality of the work. The alternative solution was to attach the code files in the submission and reference the GitHub repository.

CONTENTS

1 Introduction **1**
1.1 Motivation: Why Narrative Matters . . . 1
1.2 The Story . . . 1
1.3 From Story to System . . . 1
1.4 The Educational Tool . . . 1
1.5 Report Structure . . . 2

2 Datasets and Environment **2**
2.1 Image Dataset . . . 2
2.2 Sensor Statistics Dataset . . . 2
2.3 DungeonMazeWorld Environment . . . 3

3 Task I — Supervised Learning: Visual Entity Classification **4**
3.1 Task Description and Motivation . . . 4
3.2 Algorithms . . . 4
3.2.1 K-Nearest Neighbours . . . 4
3.2.2 Support Vector Machine . . . 5
3.2.3 Convolutional Neural Network . . . 5
3.3 Preprocessing and Design Choices . . . 5
3.4 Results and Evaluation . . . 5
3.5 Discussion and Critical Appraisal . . . 7

4 Task II — Unsupervised Learning: Behavioural Cluster Profiling **8**
4.1 Task Description and Motivation . . . 8
4.2 Algorithms . . . 8
4.2.1 K-Means Clustering . . . 8
4.2.2 Gaussian Mixture Model . . . 8
4.3 Preprocessing and Dimensionality Reduction . . . 9
4.4 Results and Evaluation . . . 9
4.5 Discussion and Critical Appraisal . . . 11

5 Task III — Reinforcement Learning: Navigation and Combat **11**
5.1 Task Description and Motivation . . . 11
5.2 Algorithms . . . 11
5.2.1 Tabular Baseline: SARSA and Q-Learning . . . 11
5.2.2 Deep Q-Network: Dueling + Double DQN . . . 12
5.2.3 Entity-Aware DQN . . . 13
5.3 Perception Integration . . . 14
5.4 Environment Enhancements and Educational Tool . . . 14
5.5 Results and Evaluation . . . 15
5.6 Discussion and Critical Appraisal . . . 16

6 Conclusion **17**

Abstract

This report presents a unified machine learning pipeline developed for the MazeDungeon scenario, in which an autonomous robotic agent, named HeroBot, must navigate a dungeon, identify hostile entities and reach the exit as efficiently as possible. Three complementary machine learning paradigms are employed: supervised learning for visual entity classification under varying artificial illumination conditions; unsupervised learning for behavioural clustering of entities from sensor statistics; and deep reinforcement learning for maze navigation and simple combat decision-making. For each paradigm, two algorithms are implemented and rigorously compared. The resulting system integrates perception, clustering and policy learning into a cohesive end-to-end pipeline. Beyond the base assignment scope, an enhanced interactive MazeDungeon environment is developed as an educational tool, enabling real-time parameter exploration and step-by-step visualisation of agent decision-making.

I. Introduction

A. Motivation: Why Narrative Matters

Before describing what was built, it is worth explaining why it was built this way. Aside from personal experiences, research in educational psychology consistently shows that narrative framing improves both engagement and retention in technical learning contexts [1, 2]. A student who is emotionally invested in an outcome, even a fictional one, even briefly, processes the underlying mechanics more actively than one presented with an abstract problem statement [3]. This is not a novel observation: it is the principle behind every serious game, every simulation-based curriculum and every case study that opens with a human story before presenting the data. The MazeDungeon scenario is an opportunity to put this principle into practice. The story that follows is simple by design. It does not need to be complex to be effective; it only needs to give the reader something to care about for the few minutes it takes to demonstrate the algorithm. The narration was kept minimal, to account for the short attention span of the modern day students.

B. The Story

HeroBot and his companion ExplorerBot are interstellar explorers, far from home, who have travelled to Earth to document its ecosystems and creatures. During one expedition, ExplorerBot is taken by a group of wild dungeon creatures who retreat into a labyrinthine dungeon under the surface. HeroBot does not panic. He prepares.

He sends RoverBot ahead: a small robotic dog, unthreatening enough to move among the creatures undetected. RoverBot's mission is data collection: images of every creature it encounters and sensor readings of their physical and behavioural characteristics. When it returns, HeroBot has two datasets to work with.

He studies them carefully. From the images he trains himself to recognise each species, even in the near-darkness of the dungeon. From the sensor readings he groups the creatures into behavioural archetypes: those that rely on brute force, those that use speed and flight and those that are intelligent and learns how to respond to each. He then builds training mazes and practices navigation: thousands of simulated traversals, learning when to fight, when to flee and how to read the environment before committing to an action.

Then he picks up his bow and his sword and heads into the dungeon.

C. From Story to System

The narrative above is not decorative. Each phase of HeroBot's preparation maps directly to a machine learning task:

- **RoverBot's image collection** → Task I: supervised classification of entity sprites under normal and dark conditions (KNN, SVM, CNN).
- **RoverBot's sensor logs** → Task II: unsupervised behavioural profiling of entities from sensor statistics (K-Means, GMM).
- **HeroBot's maze training** → Task III: reinforcement learning for navigation and combat (SARSA, Q-Learning, DQN, Entity-Aware DQN).

The three tasks are not independent exercises. They form a pipeline: the classifiers trained in Task I and the cluster model fitted in Task II are both deployed at runtime inside the Task III agent, giving HeroBot a perception system that feeds directly into his decision-making policy. This integration is described in Section 5.

D. The Educational Tool

Running in parallel with the research deliverables, an enhanced and interactive version of the MazeDungeon environment was developed. The motivation draws directly from the educational literature cited above: passive observation of an algorithm's output is a weak learning signal; active manipulation of its parameters is a strong one [3]. The enhanced environment includes a real-time visual interface, a step-by-step decision log, a perception panel showing the agent's classification probabilities at each sensing range, entity sprites rendered inside the grid, a manual control mode and a greedy-policy replay mode. The intent is that a lecturer could open this tool in front of a class and let students interact with it directly, adjusting reward values, watching the policy change, taking manual control of HeroBot and

experiencing the perception-combat loop firsthand. The concept is inspired by interactive neural network visualisers that expose internal parameter flow to the learner in real time [4]. This report documents both the research work and the tool and Section 5 describes the implementation in full.

E. Report Structure

The remainder of this report is organised as follows. Section 2 describes the datasets and the base environment. Sections 3–5 present the three tasks in full, each covering task motivation, algorithm choice, implementation, design decisions, results and critical appraisal. Section 5 describes the integrated end-to-end pipeline and the educational tool. Section 6 summarises findings and reflects on limitations and future directions.

II. Datasets and Environment

A. Image Dataset

The visual classification task uses a set of pixel-art sprite images showing five dungeon creature species that HeroBot may encounter: *human*, *halfling*, *lizard*, *orc* and *wingedrat*. Each image is an 80×80 pixel RGB PNG rendered from a top-down view. Filenames encode both species and pose (e.g. `human_A001.png`, `lizard_B001.png`), covering attack stances and directional facings across animation frames. Because each creature has multiple poses, the dataset contains meaningful within-class variation that the classifier must handle. Later it will be proven that this artificial dataset will enable certain algorithms to perform better for this specific problem, but through literature research and logical arguments, this study tries to keep in mind the real world applications 3. Representative samples from each species are shown in Figure 1.

Table 1: Image dataset class distribution.

Species	Images	Archetype
human	3,220	Smart
halfling	1,326	Smart
lizard	1,504	Tank
orc	1,383	Tank
wingedrat	585	Flying
Total	**8,018**	—

One important property of this dataset is its class imbalance: the *human* class contains 3,220 images while *wingedrat* contains only 585, a ratio of roughly 5.5:1. A classifier trained without accounting for this could achieve misleadingly high accuracy simply by predicting the majority class for every input. To prevent this, the train/test split is performed with stratification, which preserves class proportions in both partitions [16]. Per-class accuracy is reported alongside overall accuracy to make any majority-class bias visible.

The dataset plays two roles in the pipeline. At the perception level, it trains the classifiers that identify creature species from camera images. At the tactical level, species identity is passed to the Task II GMM, which assigns a combat archetype and drives the RL agent's weapon choice in Task III. Correct visual classification is therefore the first link in the chain from raw pixel input to informed combat decision.

A key challenge introduced by the dungeon setting is illumination change. At 2-block range, images are gamma-darkened using the power-law transformation $p' = p^{\gamma}$, $\gamma \sim \text{Uniform}(1.5, 12)$ [18]. This non-linear operation compresses midtones aggressively while leaving near-white pixels almost unchanged, producing images that look similar to low-light camera footage. The effect of the darkening at extreme values of γ is shown in Figure 2 and its impact on classifier robustness is examined in Section 3.

B. Sensor Statistics Dataset

The behavioural clustering task uses a tabular dataset of biometric sensor readings from 10,000 dungeon entities. In the narrative, this is the data RoverBot gathered during its scouting mission: physical and behavioural measurements recorded passively through onboard sensors. The dataset has 10,000 rows and 11 feature columns; species labels are present but deliberately excluded from the clustering pipeline.

Table 2: Sensor statistics dataset: features and missing values.

Feature	Type	Missing
stench	continuous	25
sound	continuous	30
intelligence	continuous	28
weight	continuous	34
height	continuous	34
strength	continuous	28
heat	continuous	41
flight	binary	0
magic	binary	0
bribe	continuous	0
species	label	—

Species are perfectly balanced at 2,000 entities per class, removing the class imbalance concern present in the image dataset. Missing values are sparse as table 2 suggests, fewer than 220 entries in total, under 0.25% of all data and are handled by mean imputation before scaling [17]. This approach replaces each missing value with the column mean, making it appropriate when missingness is sparse and not systematically related to the target variable, as is the case here.

Two features are excluded from the clustering

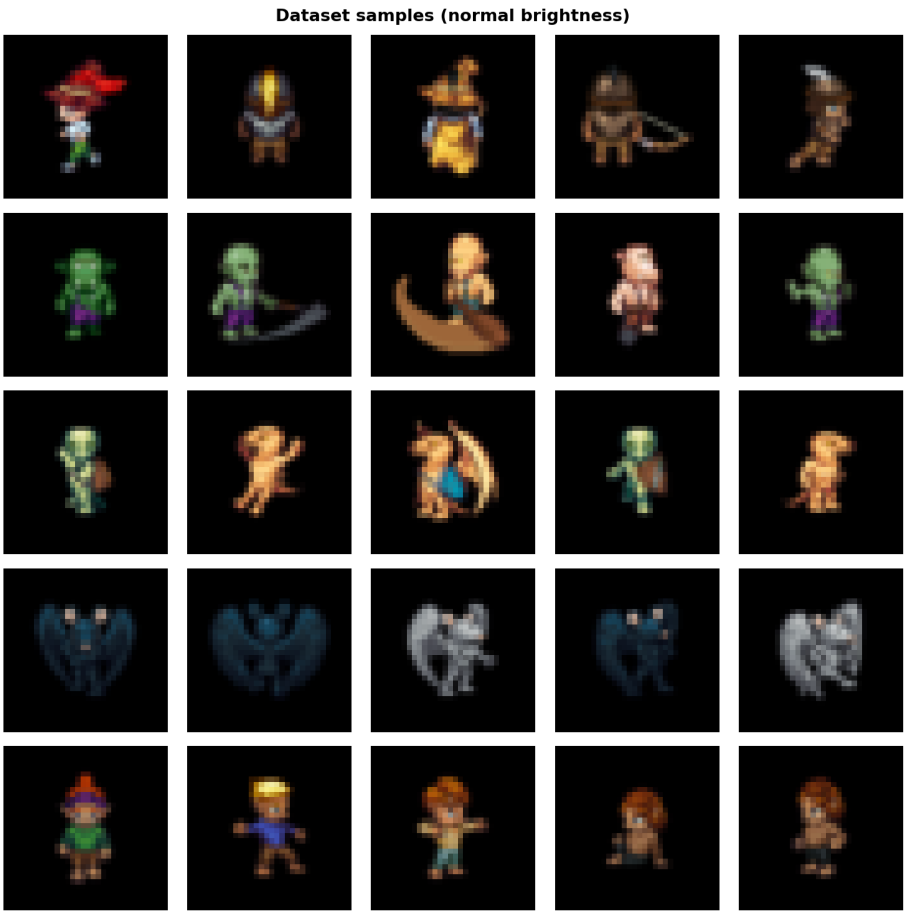

Figure 1: Sample sprites from each of the five species in rows (top to bottom: human, orc, lizard, wingedrat, halfling) at native 80×80 resolution. Multiple poses per species are shown, reflecting the animation-frame variation present in the training data.

pipeline on principled grounds. The `magic` feature contains 9,500 zero values (95% of rows), making it a near-constant flag with almost no discriminatory signal across the population. The `bribe` feature is bimodal (median = 0, max = 7,054) and encodes an economic attribute with no clear link to combat behaviour. Keeping either would distort the PCA projection without adding useful structure/information. Their exclusion is a deliberate design choice discussed further in Section 4.

One feature deserves special attention: `flight`. Approximately 29.6% of entities (2,961 of 10,000) carry `flight` = 1 and notably around 48% of lizards have this flag despite belonging mainly to the tank behavioural cluster. This apparent contradiction; flight-capable lizards clustering with brute-force tanks rather than with wingedrats, is one of the more intresting findings of the clustering analysis and is examined in Section 4.

C. DungeonMazeWorld Environment

The reinforcement learning task is built on a custom `Gymnasium` [15] environment implementing a procedurally generated 2D maze. Two variants are used across the Task III architecture.

Navigation Environment (`DungeonMazeEnv`)

The base environment gives HeroBot an $N \times N$ grid maze, where N is configurable. The agent receives a 10-dimensional normalised state vector encoding its current position, heading, three binary wall sensors (left, front, right), target position and relative displacement toward the goal. Three discrete actions are available: `turn_left`, `turn_right` and `move_forwards`. Each step costs -0.1, wall collisions cost -0.5 and reaching the goal delivers a terminal reward of $+100$.

Entity Combat Environment (`DungeonMazeEntityEnv`)

The combat variant adds three further discrete actions; `flee`, `use_bow`, and `use_sword` and expands the state to 17 dimensions: the original 10 navigation features, three entity-type probabilities at 1-block range, three at 2-block range and a

normalised energy value. Three entity types are placed per episode at fixed positions in the training maze (seed 77): a `TankEntity` at $(6, 4)$, a `SmartEntity` at $(3, 4)$ and a `FlyingEntity` placed automatically. Both fixed positions were chosen deliberately so that the agent must pass through corridors containing each entity before it can reach the exit, making avoidance impossible and combat decisions unavoidable. Also, this ensures that at least 2 detection methods are used23.

An energy budget constrains each episode. HeroBot starts with 100 energy units; every action decreases it and the episode ends at zero. This forces the agent to balance correctness against efficiency, it cannot afford to try every combat action to find the right one. A line-of-sight rule further complicates the 2-block sensor: it returns zero if a wall or entity blocks the view. This creates corner-spawn situations where the agent must move closer before it can identify the threat, making the prepared-weapon bonus non-trivial to earn.

Table 3: Reward structure for `DungeonMazeEntityEnv`.

Event	Reward
Per step	-1 energy
Wall collision	-0.5
Correct combat	$+20$, -2 energy
Wrong combat	-3, -10 energy
Wasted combat	-0.5, -3 energy
Prepared-weapon bonus	$+5$
Goal reached	$+100$ + energy bonus

One important distinction concerns how mazes are generated across the two environment variants. The tabular agents (SARSA and Q-Learning) train on a freshly generated random maze each episode, exposing them to varied layouts at the cost of a larger effective state space. Both DQN variants train on a single fixed maze (seed 42 for the navigation DQN, seed 77 for the entity DQN), trading generalisation for stable convergence. This choice and its consequence of zero success on unseen maze seeds, is acknowledged as a limitation in Section 5.

III. Task I — Supervised Learning: Visual Entity Classification

A. Task Description and Motivation

HeroBot's camera returns a 32×32 pixel RGB image when facing a cell. At 1-block range the image is captured at full brightness; at 2-block range it is gamma-darkened ($p' = p^{\gamma}$, $\gamma \sim \text{Uniform}(1.5, 12)$) [18], simulating the dim lighting of the dungeon interior. The classifier must map this image to one of five species: *orc*, *lizard*, *halfling*, *human*, or *wingedrat*. That species label is then passed to the Task II GMM, which converts it to a combat archetype and drives the RL agent's weapon choice in Task III.

Three algorithms are compared at increasing representational complexity, KNN (memory-based), SVM (margin-based) and CNN (feature-learning), to show that robustness to illumination change requires *learned* representations of the CNN, not pixel distances or linear margins. Figure 2 illustrates the effect of gamma darkening at several extreme values of γ.

Figure 2: Gamma darkening at $\gamma \in \{10, 20\}$, both above the operational range $\gamma \sim \text{Uniform}(1.5, 12)$. At $\gamma = 10$ midtones collapse; at $\gamma = 20$ the image carries no recoverable structure and is effectively unclassifiable by any method.

B. Algorithms

3.2.1 K-Nearest Neighbours

KNN [7] classifies a query image by majority vote among the k nearest training examples under the L2 Euclidian distance norm in the flattened $32\times32\times3 =$ 3,072-dimensional pixel space. The entire training set is stored and queried at inference time; there is no separate training step. The hyperparameter k was selected by sweeping over $k \in \{1, 3, 5, 7, 9, 11, 15, 21\}$ on a random 80/20 stratified split, shown in Figure 3. $k = 1$ achieved the highest accuracy at 98.4%. However, this result was identified as inflated due to data leakage and is examined critically in Section 3.4.

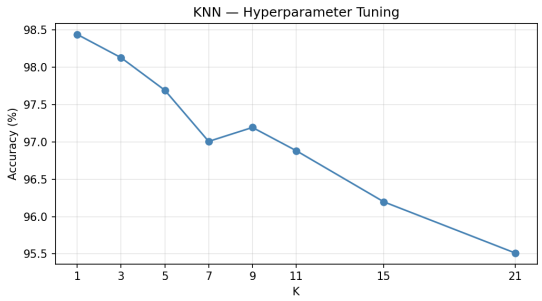

Figure 3: KNN hyperparameter sweep on the random split. All k values cluster near 98%, which is a sign of same-sprite frame leakage rather than a meaningful performance difference between k values.

The core weakness of KNN is geometric. A gamma-darkened image occupies an entirely different region of the 3,072-dimensional pixel space from its clean counterpart. The nearest-neighbour structure built on clean images is therefore destroyed by any illumination shift, regardless of k.

3.2.2 Support Vector Machine

An RBF-kernel SVM [6] was trained on the same flattened pixel representation. The kernel implicitly maps inputs to a high-dimensional feature space and finds the maximum-margin separating hyperplane within it. Hyperparameters C and γ_{rbf} were selected via `RandomizedSearchCV` [19] on a 2,000-sample subset for computational feasibility, then retrained on the full training set. `probability=True` was set to produce soft probability outputs consumed by the RL agent downstream.

Augmentation was deliberately *not* applied to SVM training. Kernel SVM scales as $\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$: tripling the dataset with stochastic dark copies would make training roughly $9\times$ slower, which was not feasible within the time constraints of this project. Since SVM is used only at 1-block range, where the image is always at full brightness, darkening robustness is never required from it, so this trade-off carries no operational cost.

3.2.3 Convolutional Neural Network

The CNN [5] takes a $32\times32\times3$ input tensor directly, without flattening. The architecture follows three convolutional blocks:

```
Conv(32) → BN → MaxPool
Conv(64) → BN → MaxPool
Conv(128) → MaxPool
GAP → Dense(128) → Dropout(0.5) →
Softmax(5)
```

Batch Normalisation (BN) [20] stabilises training and reduces sensitivity to the learning rate. Global Average Pooling (GAP) [21] replaces `Flatten`, reducing the parameter count and encouraging the network to learn spatially distributed features rather than memorising pixel positions.

The model was trained with the Adam optimiser [22] (lr $= 3\times10^{-4}$), online augmentation (zoom $\pm10\%$, horizontal flip), stochastic gamma darkening ($\gamma \sim \text{Uniform}(1.5, 12)$ per image) and early stopping on validation loss (patience = 5). Training curves are shown in Figure 4.

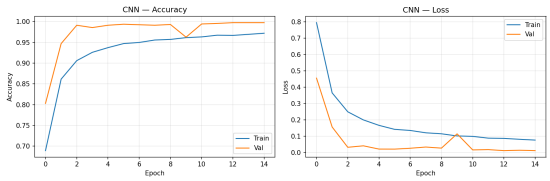

Figure 4: CNN training and validation accuracy (left) and loss (right) over epochs. Early stopping fired after validation loss stopped improving.

C. Preprocessing and Design Choices

Resizing. All images are resized from 80×80 to 32×32. This reduces input dimensionality for KNN and SVM without losing discriminative information: species identity is carried by shape and colour, not fine texture.

Normalisation. Pixel values are divided by 255, mapping inputs to $[0, 1]$. Required for gradient stability in the CNN and applied consistently across all three methods for fair comparison.

Train/test split. A stratified 80/20 split [16] preserves class proportions in both partitions. Given the 5.5:1 imbalance between *human* (3,220) and *wingedrat* (585), stratification is essential to prevent the test set, from under-representing the minority class. **Augmentation strategy.** The CNN is trained with stochastic gamma darkening ($\gamma \sim \text{Uniform}(1.5, 12)$) [18, 23]. A fixed $\gamma = 10$ was deliberately avoided: a constant darkening level would allow KNN to memorise the exact pixel transformation rather than generalise across the operational range.

SVM augmentation trade-off. Augmented training was skipped for SVM on computational grounds. The 40.1% dark accuracy reflects trained-on-clean behaviour, not an inherent ceiling of kernel methods.

D. Results and Evaluation

Primary results. Table 4 summarises accuracy under both illumination conditions. A majority-class baseline which always predicts *human*, is included as the minimum meaningful comparison [24].

Table 4: Classification accuracy under normal ($\gamma = 1$) and dark ($\gamma = 10$) test conditions (random split).

Model	Normal	Dark
Majority baseline	40.1%	40.1%
KNN ($k = 1$)	98.4%	40.1%
SVM (RBF)	99.4%	40.1%
KNN + augmentation	98.4%	95.4%
CNN (aug. trained)	99.8%	97.1%

The main result is the sharp split between augmented and non-augmented models under darkening. Both KNN ($k = 1$) and SVM collapse to the majority-class baseline at $\gamma = 10$, without exposure to augmentation. The augmented CNN retains 97.1% accuracy under stochastic darkening.

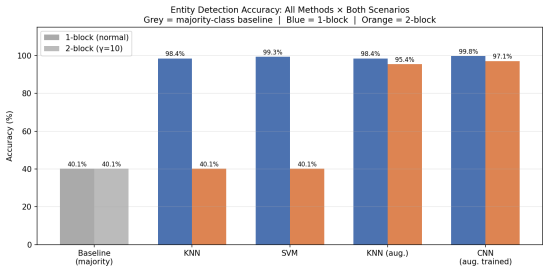

Figure 5: Accuracy under normal and dark conditions for all models. Both KNN and SVM collapse to the 40.1% majority baseline under $\gamma = 10$ darkening.

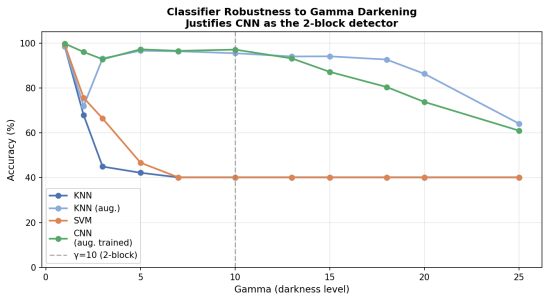

Figure 6: Accuracy vs. γ for all models. Augmented models drop at $\gamma = 2$–4 before recovering. CNN degrades more smoothly than augmented KNN across the full range.

Data leakage analysis. The dataset contains multiple animation frames per sprite identity (e.g. `human_A001.png`, `human_A002.png`). A random 80/20 split distributes these frames across training and test, meaning 75.9% of test images have a near-duplicate frame from the same sprite in the training set. At $k = 1$, the nearest neighbour under L2 distance is almost always that near-duplicate, producing artificially high accuracy regardless of whether the classifier has learned anything meaningful. To quantify this, the experiment was repeated using a *group split*: all frames of a given sprite identity are assigned entirely to training or entirely to test, so the classifier is evaluated only on sprite identities it has never seen. Results are shown in Figure 7 and Table 5.

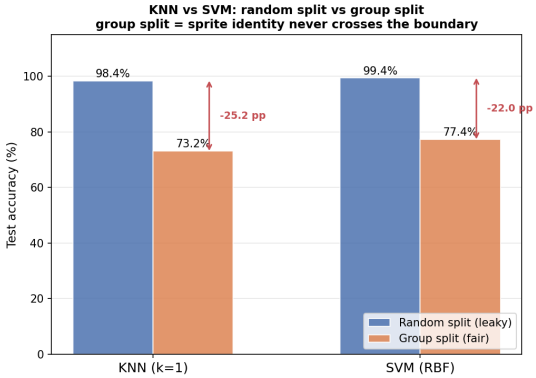

Figure 7: KNN ($k = 1$) vs SVM accuracy under random (leaky) and group (fair) splits. Both drop substantially under the group split but remain well above the 20% chance baseline, confirming that genuine class-level features are being learned.

Table 5: Accuracy under random vs group split. The drop column quantifies the inflation from same-sprite frame leakage.

Model	Random	Group	Drop
KNN $k = 1$	98.4%	73.2%	−25.2pp
KNN $k = 3$	98.1%	64.4%	−33.7pp
KNN $k = 5$	97.7%	63.4%	−34.3pp
SVM (RBF)	99.4%	77.4%	−21.9pp

The appropriate evaluation depends on the deployment question being asked. This system asks: *can the classifier recognise sprites drawn from the same asset set used in training?* The maze at runtime uses the same fixed sprite designs as the training dataset, so the classifier encounters the same sprite identities it was trained on. The random split directly models this scenario and the 98–99% figures are therefore valid for the actual deployment context. The group split answers a harder question: *would the classifier generalise to an entirely new game with different sprite designs?* That is outside the scope of this system.

Nonetheless, the group-split results (73.2% KNN, 77.4% SVM) are both well above the 20% chance baseline, confirming that both classifiers learn genuine class-discriminative features rather than purely memorising pixel duplicates. SVM's 4pp advantage under the group split is consistent with its inductive bias: the RBF decision boundary aggregates evidence across multiple support vectors rather than relying

on a single nearest neighbour, making it marginally more robust to unseen sprite variation. The gap is real but not large enough to claim SVM generalises meaningfully better to novel sprite designs. The random-split results are therefore retained as the primary evaluation metric for this system, with the group-split analysis serving as a diagnostic that honestly bounds generalisation to out-of-distribution sprites.

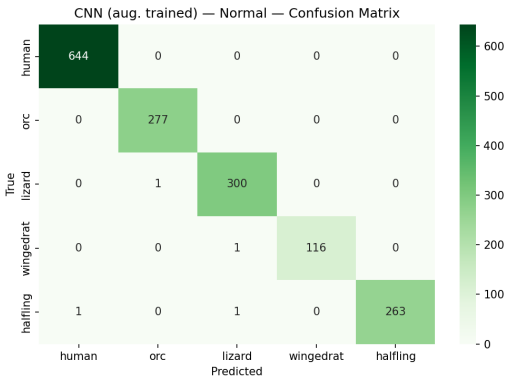

Figure 8: CNN confusion matrix on the clean test set ($\gamma = 1$). Near-perfect classification across all five species.

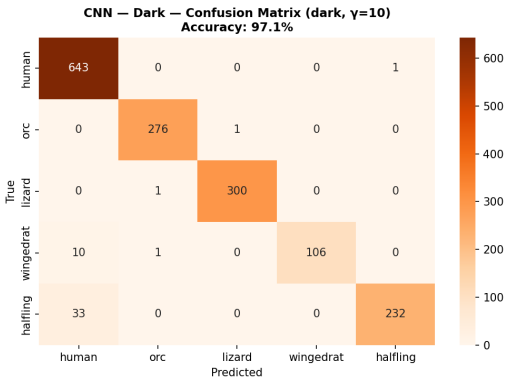

Figure 9: CNN confusion matrix on the dark test set ($\gamma = 10$). Errors concentrate in visually similar species pairs.

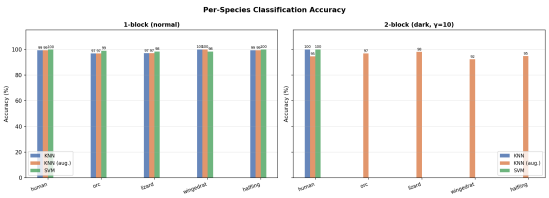

Figure 10: Per-species accuracy under normal and dark conditions. *Wingedrat* shows the highest variance, consistent with its small 585-image training set.

E. Discussion and Critical Appraisal

Model selection. The CNN at 99.8% clean and 97.1% dark is the clear best performer and is selected as the 2-block detector. The SVM at 99.4% clean is selected as the 1-block detector: it is faster at inference and darkening robustness is never required at 1-block range. Both assignments are validated empirically, the SVM correctly identifies entities in the maze at 1-block range and the CNN maintains reliable classification under the stochastic gamma darkening applied at 2-block range.

Why augmented KNN is not a genuine solution. Augmented KNN recovers from 40.1% to 95.4% dark accuracy, which looks competitive at first glance. This is an result of *transformation memorisation*: the augmented training set contains images darkened at the same γ distribution applied at test time, and KNN exploits this by matching pixel-space near-duplicates. It has not learned illumination invariance, it has memorised a specific formula. The gamma sensitivity curve in Figure 6 confirms this: augmented KNN collapses outside the training range while CNN degrades gradually and smoothly. In a real scenario with stochastic photometric variation, camera noise, partial occlusion, uneven lighting, KNN's strategy would fail entirely.

Why SVM was not augmentation-trained. Kernel SVM scaling ($\mathcal{O}(n^2)$ to $\mathcal{O}(n^3)$) makes a $3\times$ larger augmented dataset roughly $9\times$ slower to train. The augmented KNN result serves as the pixel-based comparison isolating the contribution of augmentation from the choice of classifier.

Limitations. The augmentation range ($\gamma \sim \text{Uniform}(1.5, 12)$) leaves the classifier unprepared for $\gamma > 12$, visible as a drop at the upper end of Figure 6. The dataset is entirely synthetic: real camera noise, motion blur and lens distortion are absent and a production system would need domain adaptation or real-image fine-tuning. The *wingedrat* class is underrepresented (585 vs. 3,220 images), and its dark fur makes it nearly invisible under high gamma darkening; class-weighted loss or targeted oversampling would address this. Finally, the group-split analysis shows that generalisation to entirely new sprite designs sits at 73–77% for KNN and SVM, sufficient enough for this system's fixed asset set, but a meaningful constraint for any future deployment with different sprite artwork.

Role in the pipeline. SVM provides the 1-block classification (full brightness, 99.4% accuracy, low latency). CNN provides the 2-block classification (gamma-darkened image, 97.1% accuracy, at a -5 energy cost per scan). Both feed continuous probability vectors into the RL agent's 17-dimensional state, enabling the prepared-weapon bonus described in Section 5.

IV. Task II — Unsupervised Learning: Behavioural Cluster Profiling

A. Task Description and Motivation

Before entering the dungeon, HeroBot needs to understand not just what each creature looks like, but how it behaves in combat so that it can interact with it properly. No ground-truth combat labels exist, there is no dataset of "entity X is defeated by weapon Y". This absence of supervision makes unsupervised learning the right choice: the goal is to discover hidden behavioural structure from sensor measurements alone, without any labelled training signal [10].

The source data is `dungeon_sensorstats.csv`: 10,000 entities described by biometric sensor readings collected by RoverBot. The task is to group this population into a small number of interpretable archetypes that can be named and acted upon and to save the fitted pipeline so that the Task III RL agent can classify new entities at runtime without re-fitting.

Two algorithms are compared: K-Means, which gives each entity a single hard cluster label, and a Gaussian Mixture Model (GMM), which gives each entity a probability of belonging to each cluster. The GMM is preferred as the final model for the reasons given below.

B. Algorithms

4.2.1 K-Means Clustering

K-Means [9] splits the data into k clusters by repeatedly assigning each point to its nearest centroid and updating centroids until convergence. The objective is to minimise the within-cluster sum of squares (WCSS). Clustering was applied on the 2D PCA projection rather than the raw feature space, which removes noisy dimensions and focuses sensitivity on the directions of greatest variance.

The number of clusters was chosen using two complementary criteria, shown in Figure 11. The inertia curve shows its steepest marginal drop between $k = 2$ and $k = 3$, after which additional clusters yield diminishing returns. The silhouette score [25], which measures how well each point fits its own cluster compared to neighbouring clusters (range -1 to $+1$,with higher being better), peaks at $k = 6$ (0.71), suggesting that finer structure exists in the data. However, $k = 3$ was selected on domain grounds: the RL agent has exactly three combat responses (`flee`, `use_bow`, `use_sword`) and sub-clusters beyond $k = 3$ produce archetypes with no distinct tactical interpretation. The BIC curve for the GMM (Figure 12) verifies this; the steepest improvement also occurs at $k = 3$, even though the curve continues declining through $k = 6$. This is an honest example of a domain-driven constraint overriding a purely statistical criterion.

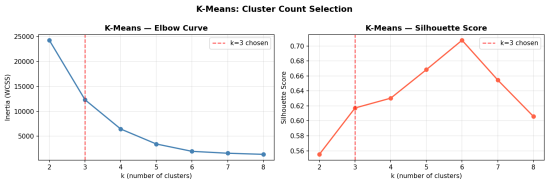

Figure 11: K-Means cluster count selection. The inertia curve (left) shows its steepest drop between $k = 2$ and $k = 3$. The silhouette score (right) peaks at $k = 6$ (0.71) but $k = 3$ is selected on domain grounds. Silhouette at $k = 3$ is 0.617, well above the random baseline of -0.007.

At $k = 3$, K-Means achieves a silhouette score of 0.617 and a Davies-Bouldin index [26] of 0.531. The Davies-Bouldin index measures the average ratio of within-cluster scatter to between-cluster separation; lower values indicate more compact and well-separated clusters. Both scores compare strongly against the random baseline (silhouette -0.007, Davies-Bouldin 97.4), confirming that the discovered structure is genuine and not an accident of the algorithm.

K-Means produces hard assignments: each entity belongs to exactly one cluster. This is a real limitation for the RL agent, which must respond to an entity that sits between two archetypes. An entity that is partly tank and partly smart gets forced into one category, losing the uncertainty signal entirely. This is the reason behind the choice of the GMM.

4.2.2 Gaussian Mixture Model

The GMM [10] models the data as a weighted sum of k Gaussian distributions, each with its own mean and covariance matrix, fitted via Expectation-Maximisation (EM). Four covariance types were compared (`full`, `tied`, `diag`, `spherical`); `full` achieved the lowest BIC [27] at 52,167 , compared to `tied` (54,417), `diag` (52,173) and `spherical` (61,947). The BIC penalises model complexity, rewarding a better fit only if it is not simply the result of adding more parameters, making it a principled criterion for selecting between covariance types. The near-identical BIC values for `full` and `diag` (difference of only 6) suggest that covariance axes are roughly aligned in PCA space, but `full` is preferred as the more general model.

The BIC curve across $k = 2$–8 is shown in Figure 12. The curve does not reach a minimum within the tested range; BIC continues declining from $k = 2$ through $k = 6$ before flattening. The steepest single drop occurs between $k = 2$ and $k = 3$ (ΔBIC $\approx$ 11,000), after which marginal improvements fall sharply, corroborating the domain-driven

choice of $k = 3$. The honest reading is that the dataset supports more than three statistically distinct sub-clusters, the GMM can find variation within species, but those sub-clusters carry no tactical meaning for the RL agent.

$k = 3$ was therefore selected on domain grounds, consistent with the three combat actions available. This is an explicit domain-driven constraint overriding the strict statistical optimum and it is acknowledged as such.

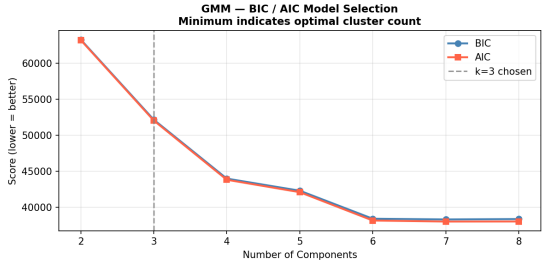

Figure 12: GMM BIC and AIC [28] across $k = 2$–8. Neither curve reaches a minimum in the tested range. The steepest drop occurs between $k = 2$ and $k = 3$ (ΔBIC $\approx 11{,}000$), after which marginal gains diminish. $k = 3$ is selected on domain grounds.

The silhouette scores of K-Means (0.617) and GMM (0.615) are nearly identical, which confirms that the cluster structure is real and stable across algorithm choice. GMM is preferred over K-Means for two practical reasons. First, soft probabilities $[p_{\text{tank}}, p_{\text{flying}}, p_{\text{smart}}]$ let the RL agent represent uncertainty, an entity assessed as 60% tank and 40% smart produces a mixed signal rather than a forced binary assignment, which maps directly into the 17-dimensional state vector in Task III. Second, the fitted GMM pipeline can classify new entities at runtime via `predict_behavior()` in `entity_behavior.py` without re-fitting, keeping Tasks II and III cleanly decoupled.

C. *Preprocessing and Dimensionality Reduction*

Feature selection. Two columns are dropped before any processing. `bribe` contains 80% zeros and encodes an economic attribute with no plausible link to combat behaviour. `magic` contains 95% zeros and is effectively a constant flag. Including either would pull the distance metric toward wealth and magic use rather than the strength, flight and intelligence axes that actually define combat strategy.

Imputation and scaling.

`SimpleImputer(strategy='mean')` replaces the sparse missing values identified in Section 2 [17]. `StandardScaler` then normalises each feature to zero mean and unit variance. This scaling step is essential: without it, high-magnitude features such as strength (≈ 85) would dominate the PCA projection over binary features such as flight ($\in \{0, 1\}$), distorting the discovered structure.

PCA. Full PCA was fitted first to produce the cumulative variance curve in Figure 13. PC1 and PC2 together capture 80.8% of total variance, which is above the 70% threshold commonly used to justify dimensionality reduction for clustering [29]. Clustering in 2D is therefore not merely a visual convenience; it retains the dominant structure of the data.

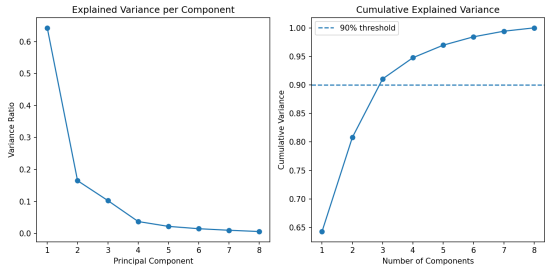

Figure 13: PCA cumulative explained variance. PC1 and PC2 capture 80.8% of total variance, justifying 2D projection for clustering.

The PCA loadings heatmap (Figure 14) shows that PC1 carries large positive weights on `stench`, `sound` and `strength`, separating physically powerful entities from intelligent ones, while PC2 is dominated by `heat` and `flight`, separating airborne entities from ground-bound ones. This loading structure already predicts the three cluster identities before clustering runs, providing strong evidence that PCA is capturing meaningful behavioural variation.

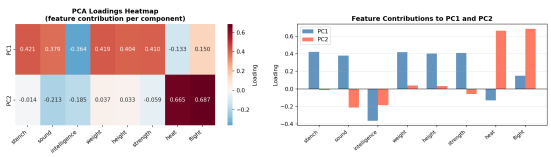

Figure 14: PCA loadings heatmap. PC1 is driven by strength, stench and sound; PC2 by heat and flight. The loading structure predicts the tank/flying/smart archetypes before clustering runs.

D. *Results and Evaluation*

Three clusters were identified and named according to their mean feature profiles and species composition, summarised in Table 6.

Table 6: Discovered cluster archetypes, dominant species and key distinguishing features.

Cluster	Species	Key features
Smart	human, halfling	High intelligence ($\approx$ 156), low strength, `flight`=0
Flying	wingedrat	`flight`=1.0, distinct heat/weight profile
Tank	orc, lizard	Highest strength ($\approx$ 85), high stench

Figure 15 shows the named cluster scatter in 2D PCA space with species markers overlaid. There is a contrast of the 5 distinct clusters in the graph shown against the three assumed categories. This has to be addressed for misinterpretation reasons; there are some subclasses which are important, such as tank humans using shields, flying lizards, or smart lizards, which do not affect the narrative and can be ignored for the generalisation process. Figure 16 shows the normalised feature radar chart, confirming that each archetype occupies a clearly distinct region of the feature space, most noticeable being the intelligence trait that stands out as the defining characteristic of smart entities. Figure 17 shows the species composition of each cluster, validating the archetype mapping with minimal cross-contamination.

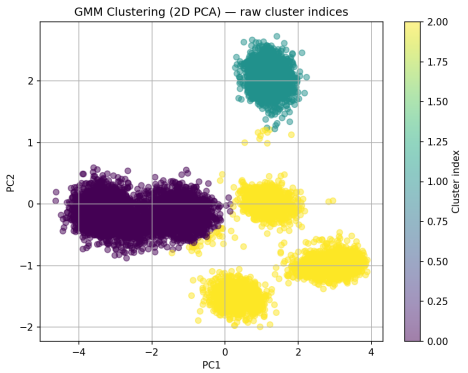

Figure 15: GMM cluster assignments in 2D PCA space with species markers. The three archetypes occupy well-separated regions, with wingedrat cleanly isolated along PC2.

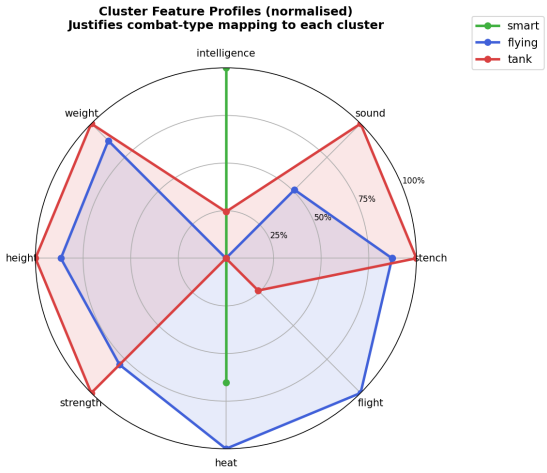

Figure 16: Normalised feature radar chart per cluster. Tank entities score highest on strength and stench; flying entities are defined by flight and heat; smart entities lead on intelligence.

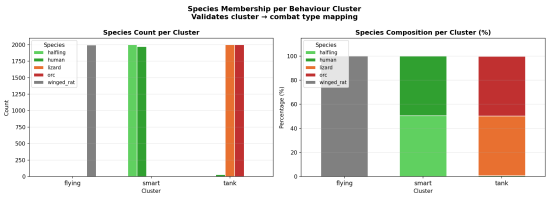

Figure 17: Species composition per cluster. Each cluster is dominated by the expected species with minimal cross-contamination, validating the archetype mapping.

One important validation finding concerns lizards. About 48% of lizards carry `flight`=1 in the dataset, yet they are placed in the tank cluster rather than the flying cluster. This confirms that the clustering is not simply recovering a single binary feature. PCA synthesises multiple features at once: lizards share strength and stench profiles with orcs strongly enough to outweigh their partial flight signal. The unsupervised approach therefore discovers structure that a simple rule-based species lookup on any single feature would miss.

Figure 18 shows the GMM assignment confidence histogram. Most entities are assigned with high confidence ($> 90\%$). The low-confidence tail consists mainly of humans sitting on the smart/tank boundary, entities whose sensor readings share traits with both archetypes. These are the most tactically interesting cases for the RL agent: a mixed $[p_{\text{tank}}, p_{\text{smart}}]$ signal should prompt a more cautious weapon choice.

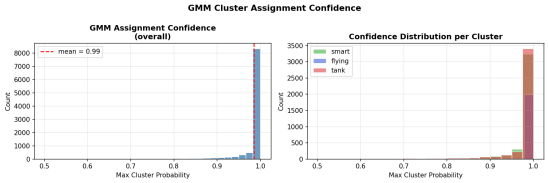

Figure 18: GMM assignment confidence histogram. Most entities are assigned with $> 90\%$ confidence. The low-confidence tail consists mainly of humans on the smart/tank boundary.

Table 7: Clustering metrics at $k = 3$. Random baseline from shuffled labels. DB: lower is better; Silhouette: higher is better.

Model	**Sil.** [25] ↑	**DB** [26] ↓	**BIC** [27]
Random	-0.007	97.45	—
K-Means	0.617	0.531	—
GMM full	0.615	0.533	52,167

Both learned models vastly outperform random assignment across all three metrics. The near-identical silhouette scores confirm that the cluster structure is real and stable across algorithm choice.

E. Discussion and Critical Appraisal

What worked. The PCA loading structure predicted the cluster identities before clustering ran, providing strong internal consistency. Species composition cleanly validates the discovered archetypes. GMM soft probabilities flow directly into the RL state vector without any translation layer. The lizard finding shows that the unsupervised approach discovers non-trivial multi-feature structure that a single-feature rule would miss entirely.

Limitations. The dataset is synthetic: sensor readings were generated from predefined species templates, so clean separation was structurally expected. On real sensor data with noise and within-species variation, cluster separation would likely be weaker. The 2D PCA projection retains 80.8% of variance but discards the remaining 19.2%: any cluster structure that only appears in PC3 or PC4 would be invisible to this pipeline. GMM assumes Gaussian-distributed clusters; multimodal or skewed distributions in PCA space would introduce bias into the soft assignments. Finally, $k = 3$ was chosen to match the three combat actions available to the RL agent. This is partly a design constraint, not a purely data-driven decision: BIC would suggest a different k on a different dataset. The honest framing is that Task II provides a data-supported justification for a choice that was partly pre-determined by the task brief.

Connection to Task III. The GMM output defines the three entity types the RL agent encounters. The cluster labels (tank, flying, smart) map directly to the three combat actions (flee, bow, sword) via the reward structure in Table 3. The fitted pipeline, imputer, scaler, PCA, GMM, is saved to `gmm_artefacts.pkl` and loaded at runtime by `predict_behavior()` in `entity_behavior.py`, so the RL agent never re-fits the model. This keeps Tasks II and III decoupled while ensuring perception and policy share a common behavioural communication.

V. Task III — Reinforcement Learning: Navigation and Combat

A. Task Description and Motivation

HeroBot must navigate a 12×12 maze from a fixed start to an exit while encountering three combat entities: tank (defeated by `flee`), flying (defeated by `use_bow`) and smart (defeated by `use_sword`). Using the wrong combat action costs energy and gives a negative reward; the correct action yields $+20$. The task is a sequential decision problem under uncertainty: the agent receives no ground-truth entity labels at runtime, only noisy perceptual signals and must act efficiently under an energy budget that ends the episode at zero.

Reinforcement learning is the right paradigm here because no labelled dataset of correct actions exists, the agent must discover the optimal policy through interaction with the environment [14]. Supervised learning cannot be applied without a pre-existing oracle policy to imitate and the sequential, delayed nature of rewards makes the problem ill-suited to single-step prediction.

The task is structured as a progressive complexity study. Tabular methods establish a baseline and expose their scalability limits. A Dueling Double DQN then replaces the lookup table with a neural approximator. Finally, the entity-aware DQN adds combat, energy management and perception, integrating the outputs of Tasks I and II directly into the agent's state and reward structure.

B. Algorithms

5.2.1 Tabular Baseline: SARSA and Q-Learning

Both SARSA [14] and Q-Learning [14] are model-free temporal-difference methods that learn $Q(s, a)$ values from experience. The key difference lies in the update target:

Q-Learning (off-policy):

$$Q(s,a) \leftarrow Q(s,a)+\alpha\big[r+\gamma \max_{a'} Q(s',a')-Q(s,a)\big] \tag{1}$$

SARSA (on-policy):

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big] \tag{2}$$

Q-Learning updates toward the maximum future value regardless of the action actually taken, mak-

ing it more aggressive and prone to overestimation. SARSA updates toward the value of the action that will actually be taken next, making it more conservative and stable in stochastic or sparse-reward environments.

Both agents were trained across five grid sizes (8×8 to 16×16) for 1,500 episodes each on a shared random seed for fair comparison. Results are shown in Table 8 and Figure 19. Learning curves at three representative grid sizes are shown in Figure 20.

Table 8: SARSA vs Q-Learning success rate (%) across grid sizes. Both collapse above 12×12.

Grid	SARSA	Q-Learning
8×8	67.5%	75.0%
10×10	47.0%	50.5%
12×12	30.0%	27.5%
14×14	18.0%	16.5%
16×16	13.5%	10.5%

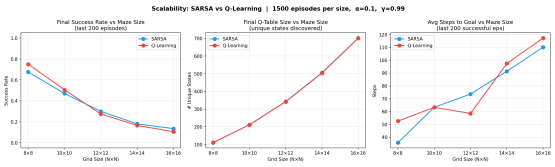

Figure 19: SARSA vs Q-Learning success rate across five grid sizes. Both methods perform poorly above 12×12, motivating the move to function approximation. Q-Learning leads at small scales; SARSA edges ahead at large scales, consistent with off-policy bootstrapping becoming unstable in sparse-reward, underexplored state spaces.

Q-Learning outperforms SARSA at small scales (8×8: 75% vs 67.5%) but falls behind at large scales (16×16: 10.5% vs 13.5%). This crossover is theoretically consistent: off-policy bootstrapping towards $\max Q(s', a')$ introduces a positive bias that accelerates learning when the state space is well-explored, but creates instability when most Q-values are at their initialised value and the maximum is uninformative. SARSA's conservative on-policy updates degrade more gracefully as the state space grows.

The root cause of collapse at scale is the Q-table itself. The table grows as $|S| \times |A|$: at 16×16 with directional heading, the state space is too large to visit meaningfully in 1,500 episodes, a direct instance of the curse of dimensionality [30]. The majority of Q-values remain at initialisation, so the agent acts effectively at random in large regions of the maze. The 16×16 results (10.5–13.5%) set the target that DQN must exceed.

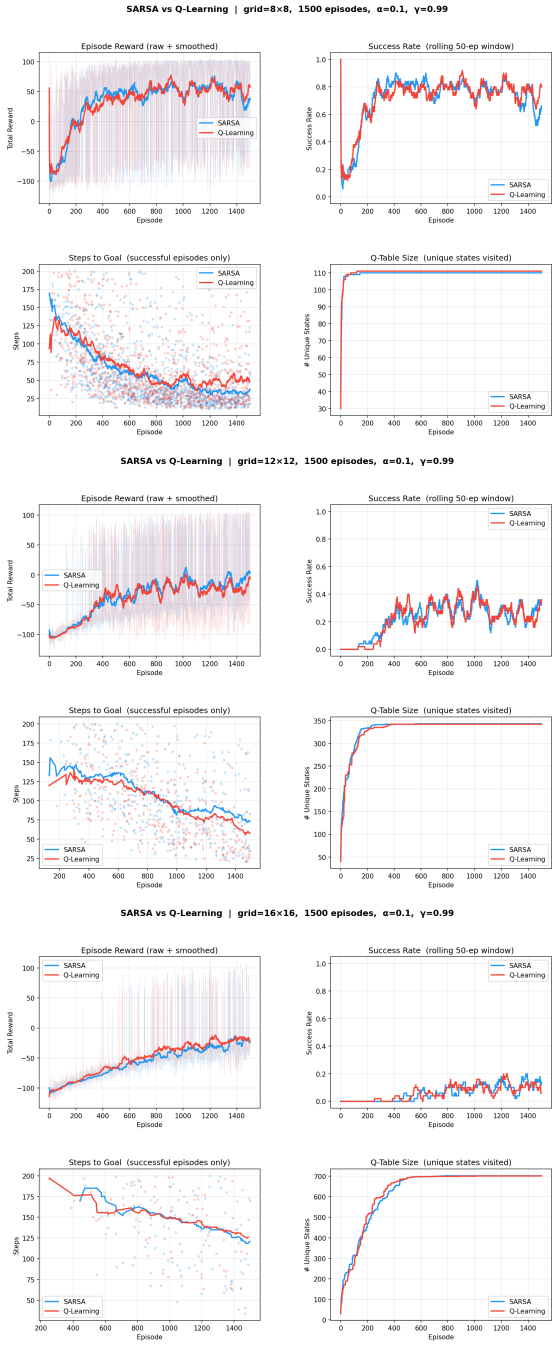

Figure 20: SARSA vs Q-Learning learning curves at three representative grid sizes. *Top:* 8×8: both methods converge reliably, Q-Learning faster. *Middle:* 12×12: success rates plateau well below 100%, Q-Learning begins to show instability. *Bottom:* 16×16: both methods fail to converge within 1,500 episodes, confirming the curse of dimensionality for tabular methods.

5.2.2 Deep Q-Network: Dueling + Double DQN

DQN [11] replaces the Q-table with a neural network that approximates $Q(s, a)$ over a continuous state space, generalising across similar states rather than memorising individual entries. The parameter count

stays constant regardless of maze size, eliminating the scalability problem of tabular methods. Transitions are stored in an experience replay buffer [11] and sampled randomly during training, which breaks the temporal correlations between consecutive steps and stabilises gradient updates.

Architecture. A Dueling DQN [12] splits into two streams after shared dense layers:

- *Value stream* $V(s)$: how good is this state, regardless of action.
- *Advantage stream* $A(s, a)$: how much better is each action relative to the average.

Recombined as:

$$Q(s, a) = V(s) + \left[A(s, a) - \overline{A}(s, \cdot)\right] \tag{3}$$

This decomposition helps navigation: most states have one clearly correct action (move forward when the path is clear) and the agent does not need to re-estimate the state value separately for each action.

Double DQN [13] decouples action selection from action evaluation to suppress Q-value overestimation:

$$a^* = \arg\max_a Q_{\text{online}}(s', a) \tag{4}$$

$$y = r + \gamma \cdot Q_{\text{target}}(s', a^*) \tag{5}$$

Without this, the $\max$ operator introduces a positive bias that inflates Q-values and destabilises training, particularly in early episodes when estimates are noisy.

The architecture is shown in Figure 21, following the design in [12].

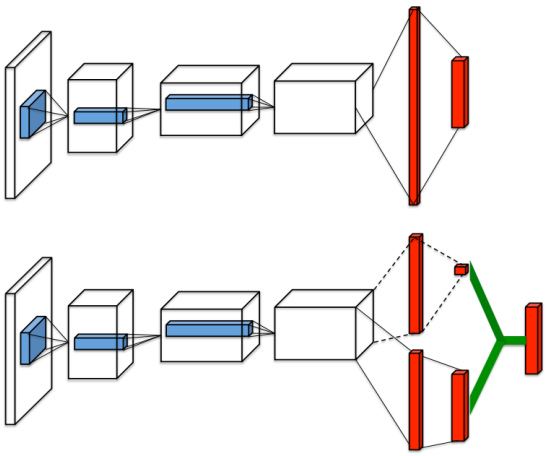

Figure 21: Dueling DQN architecture. The shared encoder splits into a value stream $V(s)$ and an advantage stream $A(s, a)$, recombined to produce Q-values.

State representation (10-D). The observation vector encodes: (x, y) position and target position normalised by grid size, heading normalised to $[0, 1]$, three binary wall sensors (left, front, right) and relative $(\Delta x, \Delta y)$ displacement to the exit. The relative offset is the important addition over a position-only state: it gives an explicit signal toward the exit from the first episode, substantially accelerating early learning.

Target network. Hard copy every 50 episodes. A soft Polyak update ($\tau = 0.005$ per gradient step) was tested first and failed: with 300 gradient steps per episode, $\tau = 0.005$ transfers effectively full weights within approximately two episodes, eliminating the stability benefit of a separate target network. Hard updates at fixed intervals preserve the required lag between online and target networks.

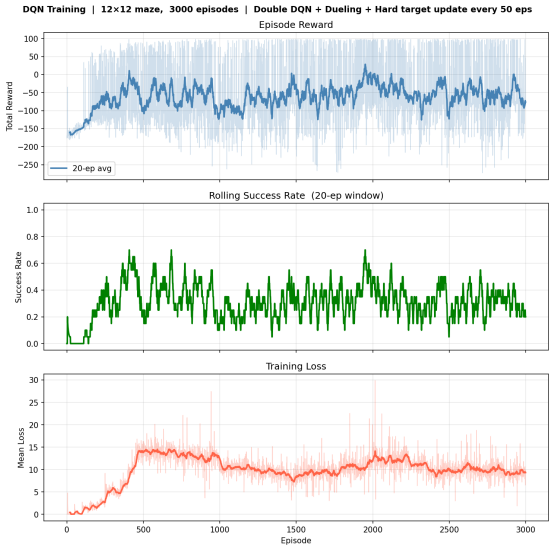

Figure 22: DQN training curves at 16×16: cumulative reward, success rate and loss over episodes. DQN achieves $\approx 31\%$ success in the final 200 episodes, compared to 13.5% for the best tabular method at the same scale.

DQN achieves approximatly 31% success at 16×16 in the final 200 episodes, more than double the best tabular result (13.5% SARSA), confirming that function approximation generalises across the state space where lookup tables fail.

5.2.3 Entity-Aware DQN

The entity-aware DQN extends the base navigation agent with combat mechanics, an energy constraint and a two-range perception system. It represents the full integration of Tasks I, II and III.

Action space (6). `turn_left`, `turn_right`, `move_forwards`, `flee` (beats tank), `use_bow` (beats flying), `use_sword` (beats smart).

State vector (17-D).

Table 9: Entity-aware DQN state vector.

Dims	Content	Notes
0–9	Navigation	Same as base DQN
10–12	1-block entity	$[p_{\text{tank}}, p_{\text{fly}}, p_{\text{smart}}]$
13–15	2-block entity	$[p_{\text{tank}}, p_{\text{fly}}, p_{\text{smart}}]$
16	Energy	energy / 100

Energy system. The agent starts each episode with 100 energy units. Costs: movement -1, correct combat -2, wrong combat -10, wasted combat (no entity present) -3, CNN scan -5. Reaching the exit awards $+100 + (\text{energy}/100) \times 20$. The episode ends at zero energy. This forces the agent to balance correctness against efficiency, whereas mindless exploration and wrong combat decisions are both penalised.

Prepared-weapon bonus. If an entity is detected at 2-block range and the agent subsequently uses the correct weapon, it earns an additional $+5$ on top of $+20$. This directly rewards using the 2-block sensor to commit to the right weapon in advance, connecting the perception pipeline to the reward signal.

Training setup. Fixed 12×12 maze, seed 77, 5,000 episodes. Tank at $(6, 4)$, smart at $(3, 4)$, flying auto-placed. Oracle mode (ground-truth binary entity flags) was used during training for stability and speed. Perception integration is evaluated separately at replay time.

C. Perception Integration

The perception pipeline connects Tasks I and II to the RL agent at runtime. The state vector shape is identical in oracle and perception modes, so trained weights transfer without retraining.

1-block range. SVM (99.3% accuracy) classifies the full-brightness adjacent image. No energy cost. The high-confidence classification is available immediately.

2-block range. CNN (97.1% accuracy on clean images, lower under stochastic darkening) classifies the gamma-darkened image at a cost of -5 energy per scan (first detection only, cached per episode). The noisier signal motivates the prepared-weapon bonus: the agent is rewarded for committing to a weapon based on an uncertain belief, encouraging advance planning despite the classification noise.

Cluster mapping from Task II GMM. The 5-class classifier outputs are aggregated into 3 cluster probabilities:

$$p_{\text{tank}} = P(\text{orc}) + P(\text{lizard})$$
$$p_{\text{flying}} = P(\text{wingedrat})$$
$$p_{\text{smart}} = P(\text{human}) + P(\text{halfling})$$

These slot directly into dimensions 10–15 of the 17-D state vector, completing the pipeline from raw pixel input to informed combat decision.

Oracle to perception transfer. Evaluated over 200 episodes: oracle-trained, oracle-evaluated achieves 100% success. Oracle-trained, perception-evaluated achieves 82%. The 18% gap comes from CNN classification noise at 2-block range: misclassification causes the agent to prepare the wrong weapon, wrong combat triggers a -10 energy penalty and energy depletion ends the episode before the exit is reached. The system degrades gracefully rather than catastrophically.

D. Environment Enhancements and Educational Tool

The base environment was extended into an interactive educational tool, motivated by the principle stated in Section 1: active manipulation of parameters is a stronger learning signal than passive observation [3]. The enhancements are a contribution beyond the assignment scope.

Visual replay. Figure 23 shows the enhanced replay interface running on seed 77. Actual creature sprites replace coloured rectangles. The right panel provides a real-time dashboard showing step count, reward, energy bar, last action and the active perception panel. When the agent is 2 blocks from an entity, the CNN scans a darkened sprite; at 1-block range the panel switches automatically to the SVM classifier and a full-brightness sprite. The entity tracker lists all three entities with their positions, alive status and correct defeat action.

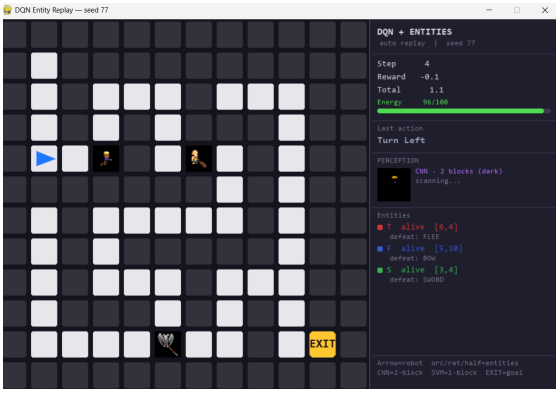

Figure 23: Enhanced replay interface (seed 77, step 4). *Left:* the 12×12 maze with entity sprites and the yellow exit cell. The blue arrow is HeroBot. *Right:* real-time dashboard showing step count, reward, energy bar ($96/100$), last action and active perception panel. The entity tracker lists positions and correct defeat actions. The status bar confirms the classifier-range assignment: CNN at 2-block, SVM at 1-block.

Terminal narration. Every CNN scan, SVM confirmation and combat outcome is logged with probabilities and result (Figure 24). This makes the connection between classifier output and agent behaviour

explicit for a student observing the system for the first time.

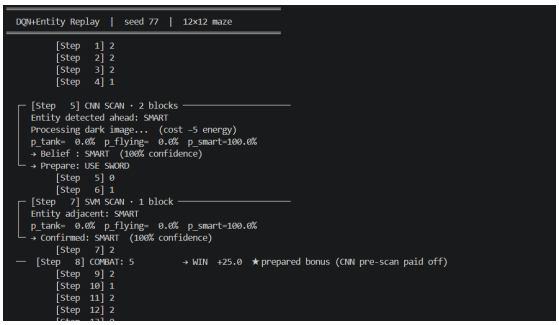

Figure 24: Terminal narration output during replay. The agent's perception pipeline, belief state and combat decision are logged step-by-step.

Manual control mode. Identical visual style to replay. The perception panel shows what a human player sees at each distance, darkened sprite at 2 cells, clear sprite when adjacent, teaching the 1-block/2-block concept through direct play rather than observation.

E. Results and Evaluation

Training progression.

Table 10: Entity-aware DQN training milestones and final performance (oracle mode, fixed 12×12 maze, seed 77).

Metric	Value
First success episode	2,234
Breakthrough (50% rolling)	2,375
Success rate (last 200 eps)	93.5%
Success rate (last 50 eps)	100%
Mean steps on success	67.7
Mean energy remaining	21.6/100
Mean combat wins per episode	2.00
Wrong combat actions	0.045
Wasted combat actions	1.055

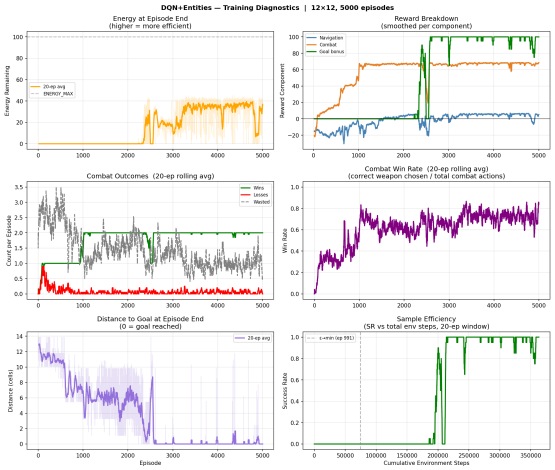

Figure 25: Entity-aware DQN training diagnostics across 5,000 episodes (12×12, seed 77, oracle mode). *Top-left:* energy remaining. *Top-right:* reward decomposed into navigation, combat and goal-bonus components. *Middle-left:* combat outcome counts. *Middle-right:* combat win rate. *Bottom-left:* distance to goal at episode end. *Bottom-right:* success rate vs. cumulative steps, showing the ε-min threshold at episode 991.

The six diagnostic panels in Figure 25 tell a coherent story of three learning phases.

Phase 1: Exploration (episodes 0–991). The ε-greedy policy [14] drives largely random behaviour. Energy is near zero at episode end throughout, distance to goal stays above 10 cells and the goal bonus is absent. Combat losses and wasted actions dominate: the agent fires weapons at empty cells or picks the wrong weapon, draining energy before it can reach the exit. The combat win rate climbs slowly, reaching roughly 40% by episode 991 when ε hits its minimum.

Phase 2: Consolidation (episodes 991–2,375). With exploration ended and the policy now greedy, the agent consolidates partial knowledge from Phase 1. Distance to goal falls from ≈ 10 cells toward ≈ 4. Combat wins stabilise at roughly 1 per episode and losses decline, while wasted actions remain volatile. Energy remaining rises above zero for the first time, indicating occasional successful episodes. Combat reward climbs steadily while the goal bonus remains absent, confirming the agent is improving combat before it can complete a full successful trajectory.

Phase 3: Breakthrough and convergence (episodes 2,375–5,000). At episode 2,375 the agent reaches a sustained 50% rolling success rate. The transition is visible as a sharp shift across all panels: energy remaining jumps to ≈ 25–40 units, distance to goal collapses to near zero, the goal bonus stabilises at ≈ 100, combat wins lock at 2.00 per episode and wrong combat actions drop to essentially zero (0.045

mean). The combat win rate settles at $\approx$ 75–85%, with residual wasted actions ($\approx$ 1.05 per episode) from occasional speculative combat in empty cells, a minor inefficiency that does not prevent success. Convergence occurs at roughly 200,000 cumulative steps, after which success holds at or near 100% for the final 50 episodes.

The agent consistently reaches the exit with roughly 22% energy remaining, showing it learned an *efficient* policy: energy is conserved, wrong combat is avoided and the trajectory to the exit is stable.

Evaluation summary.

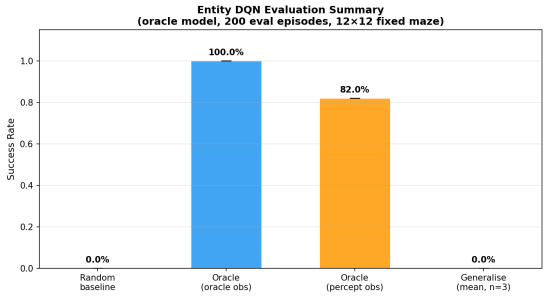

Figure 26: Entity DQN evaluation across four conditions (200 episodes each, fixed 12×12 maze). Random baseline (0%) confirms the task is non-trivial. Oracle evaluation (100%) confirms the policy is solved under training conditions. Perception evaluation (82%) quantifies the 18% cost of replacing ground-truth flags with real SVM/CNN classifiers. Generalisation (0%, mean over $n = 3$ unseen seeds) confirms memorisation rather than general navigation.

Figure 26 answers the four key evaluation questions. The random baseline rules out chance success. The oracle result confirms the policy is fully solved under training conditions. The oracle-to-perception gap isolates the perceptual cost of Task I integration: 18% of episodes fail not because navigation or combat policy is wrong, but because CNN misclassification at 2-block range triggers a -10 energy penalty that depletes the budget before the exit. The generalisation result is the most honest finding: 100% on the training maze combined with 0% on unseen seeds is clear evidence of memorisation. Both results are reported together because reporting only the 100% would be misleading.

Ablation study.

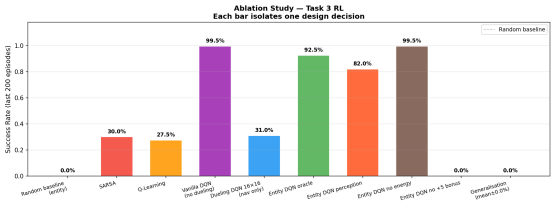

Figure 27: Ablation results: success rate (last 200 episodes) under four conditions. Removing the prepared-weapon bonus causes complete failure, confirming it is the mechanism linking perception to combat behaviour.

Table 11: Ablation results: success rate in the final 200 episodes under each condition.

Condition	Success	Interpretation
Random baseline	0%	No learning
No prepared bonus	0%	Agent ignores 2-block sensor
No energy system	99.5%	Energy not critical for success rate
Vanilla DQN	99.5%	Architecture adds stability, not accuracy
Full model	93.5%	Lower due to perception noise

The critical finding is the prepared-weapon bonus ablation. Without the $+5$ incentive the agent achieves 0% success: it never learns to use the 2-block sensor, combat decisions remain efectively random and energy depletion ends every episode early. The bonus is the mechanism connecting the perception pipeline to behaviour, without it, the entity dimensions of the 17-D state vector carry no useful learning signal and are ignored by the network.

The vanilla DQN and no-energy ablations both reaching $\approx$ 99.5% confirms that the dueling/double architecture and energy system add stability and efficiency, but the fixed maze is simple enough that simpler configurations also converge in 5,000 episodes. The full model's lower result (93.5%) reflects the additional complexity of perception evaluation mode, where CNN noise creates occasional wrong-combat failures absent in oracle conditions.

Generalisation test. The trained policy was evaluated on 3 unseen maze seeds over 200 episodes each. Success rate: 0%. Fixed-maze training produced a memorised policy, not a generalising one. This result is expected given the training setup and is discussed in the limitations below.

F. Discussion and Critical Appraisal

What the results show. The 93.5% success rate is on the same fixed maze used for training. Combined with the 0% generalisation result, this confirms

the agent memorised a specific trajectory rather than learned to navigate mazes in general. The policy is closer to a sophisticated lookup table than to a generalising RL agent. This is stated plainly because the evaluation criterion demands honest appraisal.

Why the fixed maze was used. The fixed maze isolates combat and perception as the experimental variables. Random mazes would mix navigation generalisation with combat learning, making it impossible to attribute success or failure to the perception design. Within the assignment scope this is a deliberate, justified constraint. A production system would require curriculum learning [31]: train on random mazes first, then introduce entities.

The prepared bonus finding is the most significant result in this task. It shows that without a reward signal explicitly linking advance detection to correct action, the agent ignores the 2-block sensor entirely. Integrating a classifier into an RL pipeline requires not just a state representation but a reward mechanism that makes the classifier's output actionable, this is a non-obvious design insight.

Oracle to perception gap ($100\% \rightarrow 82\%$). The 18% drop comes from CNN classification noise at 2-block range. Misclassification causes wrong combat, which triggers a -10 energy penalty and frequently depletes energy before the exit is reached. The degradation is graceful rather than catastrophic: the system fails on energy, not on navigation. Training directly on perception outputs rather than oracle flags would most likely close this gap, at the cost of slower and noisier convergence.

Limitations.

- Fixed maze training produces memorisation, not generalisation. A local observation window or recurrent network (LSTM) [32] would be needed for random-maze generalisation.
- Oracle training with perception replay is a train-test distribution mismatch. Direct perception training would close the 18% gap but require more episodes.
- Three entities on a 12×12 fixed maze is a simplified scenario. Real dungeon environments would have dynamic entity placement, multiple encounters and varied layouts.
- The model-free approach cannot plan ahead. A model-based hybrid (Dyna-Q [14] or MCTS) would be substantially more sample-efficient on a deterministic fixed environment.
- No multi-seed training variance is reported. A single training run does not confirm robustness to initialisation.

VI. Conclusion

This report presented a unified machine learning pipeline for the MazeDungeon scenario, in which HeroBot must navigate a dungeon, identify hostile entities and reach the exit efficiently. Three complementary paradigms were employed, supervised learning for visual classification, unsupervised learning for behavioural profiling and deep reinforcement learning for navigation and combat, with two algorithms implemented and compared for each.

Task I demonstrated that robustness to illumination degradation requires learned representations. KNN and SVM both collapsed to the majority-class baseline (40.1%) under gamma darkening, while the augmented CNN retained 97.1% dark accuracy by learning spatially localised features through convolution rather than memorising pixel distances. The SVM (99.3% clean accuracy) was assigned the 1-block role where full brightness is always guaranteed; the CNN was assigned the 2-block role where darkening robustness is essential.

Task II showed that three interpretable behavioural archetypes, tank, flying and smart which emerge naturally from sensor statistics without any combat labels. K-Means and GMM produced nearly identical silhouette scores (0.617 and 0.615 respectively), confirming that the cluster structure is genuine and stable across algorithm choice. The GMM was preferred as the final model because its soft probability outputs flow directly into the RL agent's state vector, preserving uncertainty rather than forcing hard assignments. The finding that flight-capable lizards cluster with tanks rather than wingedrats demonstrated that the pipeline discovers non-trivial multi-feature structure.

Task III showed that tabular methods collapse above 12×12 due to the curse of dimensionality, while a Dueling Double DQN more than doubled their performance at 16×16 (31% vs 13.5%). The entity-aware DQN reached 93.5% success on the training maze, with the ablation study revealing that the prepared-weapon bonus is the critical mechanism connecting perception to behaviour, without it, the agent ignores the 2-block sensor entirely. The most honest result in the evaluation is the 0% generalisation to unseen maze seeds, reported alongside the 100% oracle result because omitting it would be misleading.

Integration. The three tasks form a coherent end-to-end pipeline: the SVM and CNN trained in Task I feed classification probabilities into a 17-dimensional state vector, the GMM fitted in Task II maps those probabilities to combat archetypes and the RL agent trained in Task III acts on both signals simultaneously. The prepared-weapon bonus demonstrated that this integration requires not just a shared state representation but a reward mechanism that makes

the perception output behaviourally actionable.

Beyond the core deliverables, an enhanced interactive MazeDungeon environment was developed using Pygame as an educational tool. An earlier implementation using Tkinter was abandoned after rendering limitations made real-time sprite display and smooth frame updates impractical; Pygame provided the low-level display control needed for a fluid interface. The resulting tool supports visual replay of trained policies, step-by-step decision logging with classifier probabilities and a manual control mode where a user can navigate the maze directly and experience the perception pipeline firsthand.

Future Work

Several directions could substantially extend this work.

Generalisation through curriculum learning. The most immediate limitation is the 0% generalisation to unseen mazes. The natural fix is curriculum learning [31]: begin training on small random mazes without entities, progressively increase maze size, then introduce entities once navigation is stable. A local observation window or an LSTM-based [32] policy network would further help the agent handle layouts it has never seen, replacing the global position encoding that currently anchors it to the training maze.

Richer combat system. The current tank/flying/smart structure is deliberately simple, a three-way rock-paper-scissors that maps cleanly to three weapons. A more realistic combat system would include continuous health values, ranged vs melee weapon trade-offs, enemy movement and pursuit behaviour and mixed-archetype entities that cannot be cleanly assigned to a single cluster. This would also stress-test the GMM soft assignments: entities near cluster boundaries, currently a minor curiosity, would become a genuine tactical challenge requiring the agent to hedge its weapon choice.

Evolutionary and population-based approaches. The navigation and combat policy could be explored using neuroevolution or genetic algorithms [33] as an alternative to gradient-based RL. Evolutionary methods do not require differentiable reward signals, handle sparse rewards naturally and maintain a population of diverse policies that could be evaluated in parallel across different maze seeds, directly addressing the generalisation problem without curriculum scheduling.

More realistic vision and augmentation. The image dataset consists entirely of synthetic pixel-art sprites. A more realistic perception pipeline would require real or photorealistic entity images, domain randomisation across lighting conditions beyond the power-law gamma model used here and augmentation strategies such as additive noise, motion blur and partial occlusion [23]. The CNN architecture would also benefit from pretraining on a larger sprite or game-art dataset before fine-tuning on dungeon images, reducing the sensitivity to the small and imbalanced training set.

Creating Images for the story. Creating images for the characters of the story, might sound simple but having an audiovisual supplementary material that makes presentation more interesting, enhances the user experience.

From simulation to interactive game. The enhanced Pygame environment is a functional proof of concept, but the underlying system, a navigating agent with a visual perception pipeline, a behavioural clustering model and a reward-shaped combat policy, is architecturally close to a simple game AI. Given a background in video game development, a natural extension would be to port this into a fully interactive game using a dedicated engine. Unity ML-Agents [34] would be a strong candidate: it provides a native interface between a game environment and Python-based RL training loops, supports curriculum learning out of the box and would allow HeroBot's policy to be trained directly inside a richer, visually complete dungeon without the overhead of a custom Gymnasium wrapper. The manual control mode already developed suggests the game loop logic is sound; the main work would be replacing the Pygame renderer with Unity's scene graph and exposing the observation and action spaces through the ML-Agents API.

REFERENCES

[1] M. C. Green and T. C. Brock, "The role of transportation in the persuasiveness of public narratives," *Journal of Personality and Social Psychology*, vol. 79, no. 5, pp. 701–721, 2004.

[2] M. P. J. Habgood and S. E. Ainsworth, "Motivating children to learn effectively: Exploring the value of intrinsic integration in educational games," *Journal of the Learning Sciences*, vol. 20, no. 2, pp. 169–206, 2011.

[3] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge: Cambridge University Press, 2009.

[4] D. Smilkov, S. Carter, D. Sculley, F. B. Viégas and M. Wattenberg, "Direct-manipulation visualization of deep networks," *arXiv preprint arXiv:1708.03945*, 2017.

[5] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[6] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[7] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[8] C. K. I. Williams and M. Seeger, "Using the Nyström method to speed up kernel machines," in *Advances in Neural Information Processing Systems*, vol. 13, 2001, pp. 682–688.

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Prob.*, 1967, pp. 281–297.

[10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.

[11] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[12] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," in *Proc. ICML*, 2016, pp. 1995–2003.

[13] H. van Hasselt, A. Guez and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016, pp. 2094–2100.

[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[15] M. Towers *et al.*, "Gymnasium," *arXiv preprint arXiv:2407.17032*, 2024.

[16] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[17] R. J. A. Little and D. B. Rubin, *Statistical Analysis with Missing Data*, 2nd ed. Hoboken, NJ: Wiley, 2002.

[18] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York: Pearson, 2018.

[19] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[20] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. ICML*, 2015, pp. 448–456.

[21] M. Lin, Q. Chen and S. Yan, "Network in network," in *Proc. ICLR*, 2014.

[22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[23] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.

[24] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intelligent Data Analysis*, vol. 6, no. 5, pp. 429–449, 2002.

[25] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987.

[26] D. L. Davies and D. W. Bouldin, "A cluster separation measure," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[27] G. Schwarz, "Estimating the dimension of a model," *Annals of Statistics*, vol. 6, no. 2, pp. 461–464, 1978.

[28] H. Akaike, "A new look at the statistical model identification," *IEEE Trans. Autom. Control*, vol. 19, no. 6, pp. 716–723, 1974.

[29] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.

[30] R. E. Bellman, *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[31] Y. Bengio, J. Louradour, R. Collobert and J. Weston, "Curriculum learning," in *Proc. ICML*, 2009, pp. 41–48.

[32] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[33] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint arXiv:1712.06567*, 2017.

[34] A. Juliani *et al.*, "Unity: A general platform for intelligent agents," *arXiv preprint arXiv:1809.02627*, 2018.

APPENDIX A: CODE REPOSITORY AND REPRODUCIBILITY

The full source code, trained model weights, and all supporting materials are available at the following GitHub repository:

`https://github.com/argi4s/AI_in_Robotics/`

A note on code inclusion: the original intent was to embed commented code listings directly in this report. However, the volume of figures, tables and extended prose pushed the Overleaf project beyond its compilation memory limit, requiring figures and code blocks to be trimmed to keep the document buildable. Purchasing a premium Overleaf plan was not a viable option given submission time constraints. The core implementation choices sucha as architecture decisions, hyperparameter selections, preprocessing steps and reward structures are described in full in the main body and are sufficient for reproducibility from the repository.

Quick Start

All commands are run from the project root with the virtual environment active (Python 3.10 or above required).

Installation:

```
python -m venv venv
venv\Scripts\activate       # Windows
# source venv/bin/activate  # macOS/Linux
pip install -r requirements.txt
```

Watch the trained agent (recommended starting point):

```
# Full pipeline: DQN navigates, identifies
# and defeats enemies using Task 1 classifiers
# and Task 2 GMM clusters at runtime
python -m replay.replay_entity

# Navigation only, no entities
python -m replay.replay_dqn
```

Play manually:

```
# Keyboard controls:
# W=forward  A=left  D=right
# 1=flee  2=bow  3=sword
# R=reset  Q=quit
python -m manual.manual_entity
```

Reproduce training:

```
# Tabular agents (SARSA + Q-Learning,
# 5 grid sizes)
python -m agent_comparison

# Navigation DQN
python -m train.train_dqn

# Entity DQN (full pipeline)
python -m train.train_dqn_entities

# Ablation study
python -m train.run_ablation --config A
python -m train.run_ablation --config B
python -m train.run_ablation --config C
python -m train.eval_entity
python -m train.ablation_table
```

Replay speed and settings can be adjusted by editing `replay_config.py` in the project root — no other files need changing:

```
STEP_DELAY = 0.35  # seconds between moves
MAZE_SEED_ENTITY = 77  # maze layout
GRID_SIZE  = 12    # must match trained model
```

Task Integration at Runtime

When `replay_entity.py` runs, the three tasks operate as a live pipeline with no retraining required:

1. The **CNN** (Task I) scans entities at 2-block range and returns $[p_{\text{tank}}, p_{\text{flying}}, p_{\text{smart}}]$.
2. The **SVM** (Task I) confirms identity at 1-block range from a full-brightness image.
3. Both probability vectors feed into the 17-D state vector the **DQN** (Task III) receives.
4. The DQN selects the correct combat action: `flee` → tank, `bow` → flying, `sword` → smart.

The Task II pipeline is serialised to `task2/results/gmm_artefacts.pkl` and loaded at runtime by `predict_behavior()`, keeping all three tasks decoupled while sharing a common behavioural vocabulary.

APPENDIX B: USE OF GENERATIVE AI

The use of generative AI in this project was minimal and limited to tasks outside the core learning outcomes. Specifically, it was used to assist with debugging the transition from Tkinter to Pygame for the interactive replay environment and to convert text description of statistical analysis into LaTeX table formatting in order to reduce the document length. All machine learning design decisions, algorithm implementations, experimental results and written analysis are entirely the author's own work.